17/04-2026
Hold: WebH125-1

Nordic Table

Smagsoplevelser fra det nordiske køkken

Skrevet af: Oleg Troian

GitHub Nordic Table

Login
Kode: admin
Email: admin@mediacollege.dk

Indholdsfortegnels

Vurdering af egne indsats.....3
Hvad ville jeg gøre anderledes 4
Struktureret dataflow i React 5
AI-værktøjer 6
Hvordan har jeg udviklet mig 7
Erklæring 8

Vurdering af egne indsats

Dette projekt er udviklet med React og har været en lærerig proces, hvor jeg har arbejdet med opbygning af en moderne frontend-applikation.

Projektet er stylet med TailwindCSS, hvilket gjorde det muligt hurtigt at opbygge et responsivt layout baseret på designet fra Figma. Jeg har anvendt clsx til håndtering af dynamiske klasser, hvilket har forbedret kodens struktur og læsbarhed.

Applikationen er opbygget med fokus på genbrugelige komponenter som f.eks. Section, Card og Button, hvilket har gjort det nemmere at vedligeholde og genbruge kode på tværs af projektet.

Jeg har arbejdet med routing ved hjælp af React Router, herunder opbygning af sider som menu, booking og et login til admin dashboard.

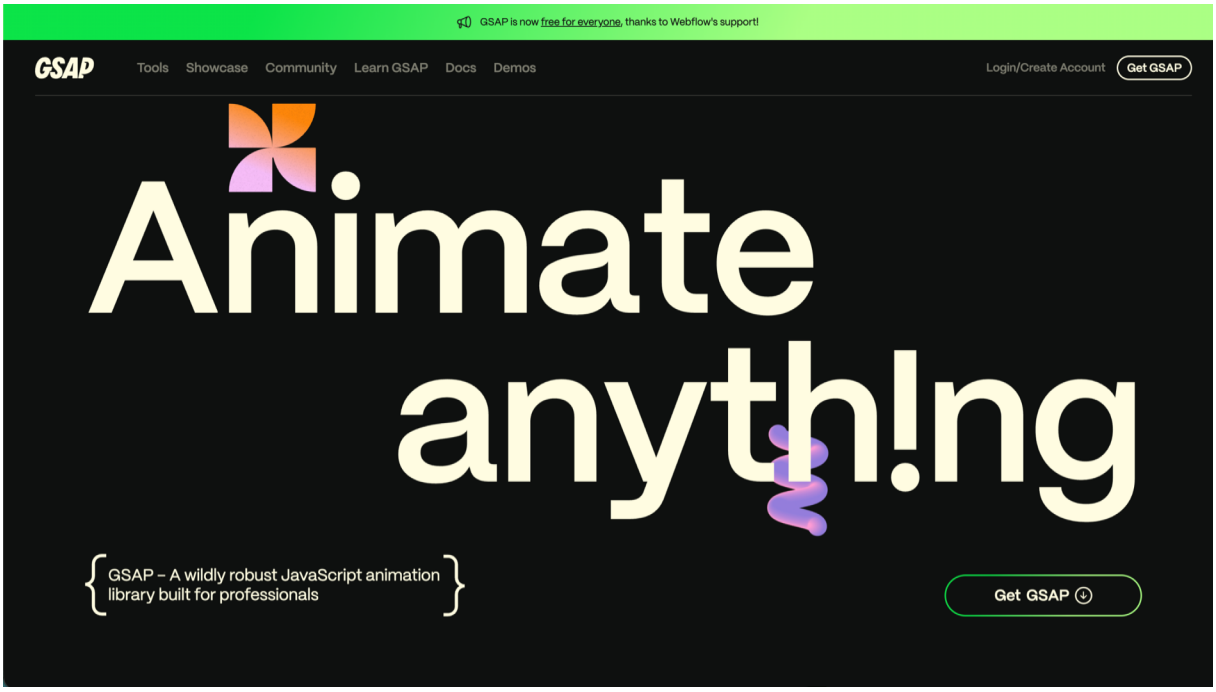
Til planlægning af projektet anvendte jeg GitHub Projects, hvilket gav et godt overblik over udviklingsprocessen.

Alt i alt er jeg tilfreds med, at jeg har omsat Figma-design til en responsiv og brugervenlig webapplikation med klar struktur.

Hvad ville jeg gøre anderledes

Hvis jeg havde haft mere tid til projektet, ville jeg have arbejdet mere med animationer og interaktive elementer for at forbedre brugeroplevelsen.

For eksempel kunne jeg have implementeret animationer til tekst og sektioner ved scroll, hvilket ville gøre siden mere levende og engagerende for brugeren. Her ville det være oplagt at anvende et bibliotek som GSAP, som giver avancerede muligheder for at styre animationer og transitions.



Derudover ville jeg have brugt mere tid på at finjustere detaljer i UI'et og skabe en endnu mere flydende oplevelse mellem de forskellige sektioner.

Struktureret dataflow i React

Under arbejdet med projektet har jeg forbedret mine færdigheder inden for datahåndtering i React samt opbygning af genanvendelige hooks.

Jeg har udviklet en generisk fetch-funktion (fetchData), som gør det muligt at hente data fra forskellige API-endpoints ved blot at angive endpoint-navnet. Funktionen håndterer både API-kald, fejl og JSON-parsing, hvilket gør koden mere genanvendelig og reducerer duplikering.

```
App.jsx    useDishes.js ×
nordic_table_front-end > src > components > Dishes > useDishes.js > ...
You, 2 days ago | 1 author (You)
1  import React, { useState, useEffect } from "react";  7.9k (gzipped: 3.1k)
2  import { fetchDishes } from "../../../utils/api";
3
4  export function useDishes() {
5    const [dishes, setDishes] = useState([]);
6    const [loading, setLoading] = useState(false);
7    const [error, setError] = useState("");
8
9    // Fetch dishes data when the component mounts
10   useEffect(() => {
11     const fetchData = async () => {
12       try {
13         setLoading(true);
14         const data = await fetchDishes();
15         setDishes(data);
16       } catch (err) {
17         setError(err.message);
18       } finally {
19         setLoading(false);
20       }
21     };
22     fetchData();
23   }, []);
24
25   // return dishes data, loading status, and error message
26   return { dishes, loading, error };
27 }
```

Derudover har jeg implementeret custom hooks, såsom useDishes, der håndterer data, loading-state og fejlbeskeder. Dette gør komponenterne mere rene og fokuserede på UI, mens datahåndteringen er isoleret i hooks.

Jeg har også arbejdet med miljøvariabler (VITE_API_BASE_URL) til at adskille API-konfiguration fra selve koden, hvilket gør applikationen mere fleksibel og nemmere at vedligeholde.

```
api.js ×
nordic_table_front-end > utils > api.js > fetchData
You, yesterday | 1 author (You)
1  // api url from .env file
2  const API_URL = import.meta.env.VITE_API_BASE_URL;
3
4  // Generic function to fetch data from a given endpoint
5  const fetchData = async (endpoint) => {
6    try {
7      const res = await fetch(`${API_URL}/${endpoint}`);
8      if (!res.ok) throw new Error(`Error fetching ${endpoint}`);
9      const data = await res.json();
10     return data.data || [];
11   } catch (error) {
12     console.error(error);
13     return [];        You, 3 days ago • Initial commit …
14   }
15 };
16
17 //? export specific functions for each endpoint
18 export const fetchDishes = () => fetchData("dishes");
19
```

Samlet set har arbejdet med fetch-funktioner og hooks styrket min forståelse af asynkron datahåndtering, React state management og hvordan man strukturerer genanvendelig frontend-kode.

AI-værktøjer

AI-baserede værktøjer er blevet anvendt som understøttende hjælpemidler under udviklingen af dette projekt.

ChatGPT blev primært anvendt til:
- ✓ hjælp med udarbejdelse af dokumentation
- ✓ oversættelse af indhold til dansk (rapport og UI-tekst)
- ✓ sparring omkring formuleringer og struktur i teksten

GitHub Copilot blev anvendt til:
- ✓ forslag til kode og forbedring af kodens læsbarhed
- ✓ assistance med Tailwind CSS og komponentstruktur
- ✓ hjælp til debugging og problemløsning

AI-værktøjerne blev udelukkende anvendt som støtte og inspiration og ikke som erstatning for selvstændig udvikling.

Grundlæggende kode og løsninger er baseret på viden og erfaring opnået gennem undervisningen og praktik. AI-værktøjer blev primært anvendt til at forbedre struktur, læsbarhed og tilpasning af koden til projektets omfang og krav.

AI kode og alle forslag er:
- ✓ gennemgået og forstået før implementering
- ✓ tilpasset projektets struktur og krav
- ✓ manuelt integreret og testet

Jeg har selv stået for strukturering af projektet, herunder opbygning af komponenter, mappestruktur og arkitektur, samt alle væsentlige beslutninger i udviklingsprocessen.

Den færdige løsning afspejler derfor min egen forståelse og mine egne valg.

Hvordan har jeg udviklet mig ?

Gennem arbejdet med projektet har jeg især udviklet mine færdigheder inden for datahåndtering i React.

Jeg har arbejdet med integration til et API, hvor jeg har hentet og vist dynamisk data såsom retter. I den forbindelse har jeg implementeret en generisk fetch-funktion samt custom hooks, som håndterer data, loading og fejl. Dette har givet mig en bedre forståelse for asynkron datahåndtering og separation of concerns.

Jeg har også arbejdet med filtrering og strukturering af data, herunder visning af "signature dishes" baseret på en boolean værdi samt gruppering af retter efter kategori.

Derudover har jeg fået erfaring med formhåndtering ved hjælp af Formik og Yup, samt håndtering af login med JWT-token.

En vigtig læring for mig har været brugen af environment variables (.env), hvor jeg har arbejdet med at adskille API-konfiguration fra koden. Dette har gjort applikationen mere fleksibel og skalerbar.

Samlet set har projektet styrket min forståelse for, hvordan man opbygger en struktureret og vedligeholdelsesvenlig frontend-applikation.

Erklæring

Jeg bekræfter hermed, at jeg selvstændigt og uden uretmæssig hjælp har udviklet det afleverede eksamensprojekt i overensstemmelse med gældende regler for prøven.

Oleg Troian